权重空间编程：方法、成本与局限

200 篇文献的跨领域中文综述

版本：2026-11-09 · 叙述性综述 · 覆盖权重空间、零阶优化、表征引导、NAS 与具身控制

摘要

直接修改神经网络内部状态，正在成为连接模型适配、可解释性与自动设计的一条研究线索。权重空间编程（Weight-Space Programming，WSP）进一步提出：能否让语言模型阅读、构造或编辑另一模型的参数，并把可执行修改交给外部评估？本文围绕这一问题，整理 200 篇论文与预印本，以干预对象、提案机制、信息权限和验证协议四条轴线，比较权重生成与知识编辑、零阶优化、激活引导、神经架构搜索及机器人策略干预。重点不是把这些方法归为同一种算法，而是解释它们如何共享接口、转移成本，又在哪里失去可比性。

综合证据显示，少量参数或激活的修改有时能显著改变行为，但局部成功不能直接证明通用的权重理解、知识一致性或长期控制能力。计算节约也常常依赖预训练模型、辅助网络、预计算统计量、参考轨迹或历史搜索数据。本文据此提出分阶段成本账本，分别记录准备、搜索、部署和独立验证，并将副作用、分布外迁移、重复试验和失败回退纳入评价。200 篇文献中，62 篇核验了全文相关章节，138 篇完成题名、元数据及摘要筛选；这些深度区别随文献矩阵公开。本文未开展新实验，也不声称穷尽检索或确立新领域的优先权。

1 问题与阅读路线

传统微调让训练算法通过损失梯度更新参数；提示工程改变输入；表征引导在前向传播中改变隐藏状态；NAS 搜索计算图。WSP 的特色在于把参数或编辑程序作为语言模型可操作的对象，使结构先验、程序生成和反馈驱动搜索进入参数修改过程。[1, 2, 4, 6, 10, 11] 这是一种具体研究设定，而不是“所有模型修改”的同义词。零阶优化不要求语言模型参与，激活引导也未必不使用梯度，NAS 更同时包含强化学习、演化与可微优化。[51, 86, 131]

文献主组	数量	本文关注的问题
权重空间与编辑 [1-50]	50	参数如何被表示、生成、组合和验证？
零阶优化 [51-85]	35	没有反向传播时，查询如何换取更新方向？
表征与激活引导 [86-130]	45	方向从何而来，何时施加，副作用如何测量？
神经架构搜索 [131-175]	45	搜索器、代理估计与最终架构表现如何区分？
具身模型与闭环评估 [176-200]	25	推理干预能否改善真实时间中的动作序列？

阅读时可先看第 3 节的共同描述方式，再按第 4 至 13 节进入各方法，最后用第 14、15 节的成本账本和实验问题组织自己的研究。表格中的成本均表示需要统计的项目；除明确标注原论文结果外，不代表本文做出的实测排名。

2 文献获取、证据深度与已有综述

2.1 检索和筛选方式

本轮检索日期为 2026-11-09。以 arXiv 元数据接口为主要检索源，执行 16 组查询，覆盖 weight space、knowledge editing、zeroth-order、activation steering、neural architecture search、vision-language-action 等词组，并结合前期核心文献及定向补检。各查询的字段、排序方式、返回上限与实际返回量记录在 search-log.csv 中。查询结果合并后有 528 条唯一 arXiv 记录；加入 77 条种子 / 补检记录后，合计 590 条候选。这里的数量是本次限定查询返回的记录数，不是数据库全部相关文献数。

按标识符合并版本，检查题名与主题，排除明显错配、非实质研究示例及范围外记录，再按方法代表性、评估价值与分支覆盖选入 200 篇。三条定向检索中的错误标识符已剔除；一篇摘要明确声明为 AI 自动生成演示的 VLA“综述”也被替换。其余未选记录并不等于质量不合格。主组只用于计数；例如 ZO-DARTS 同时连接 ZO 与 NAS，Steer2Edit 同时连接激活和权重编辑。[75, 120] 因此分组数量不能解释为领域规模或热度。

62 篇文献核验了全文中与本综述有关的方法、结果或限制章节，其余 138 篇以摘要层面的机制与范围判断为主。较新的工作包含尚未经同行评审的预印本。对于摘要筛选记录，本文只作方法定位，不将摘要中的领先幅度直接汇总成性能结论。元数据、链接、原始发布日期、核验深度及逐篇中文笔记保存在 corpus-200.csv；关键数字和推断另见 evidence-ledger.csv。可核验性不等于已经独立复现。

2.2 本文与已有综述的关系

已有综述及核验位置	已覆盖的核心内容	本文采用的连接方式
NAS 综述 [131] §2-4；补充综述 [143]	搜索空间、策略、性能估计	把代理成本与最终训练分开
ZO 入门综述 [51] §II-III	函数值访问、估计器及应用	连接模型接口和查询单位
权重空间学习综述 [43] §2-4、§5.6	理解、表示、生成及 NAS	区分专门训练的元模型与通用 LLM
RepE 综述 [86] §2-4、§6-7；分类综述 [107] §3-8、附录 B	表征读取、操作、评估	强调准备成本和干预副作用
可操作解释综述 [116] §2-5；知识编辑综述 [19] §2、§4	定位、引导、改善；可靠性与局部性	把局部干预接到独立行为验证

本文的组织价值是提供跨分支的比较接口，而非宣称这些分类轴首次出现。上述对应关系来自对既有综述相关章节的正向核验，不以“标题没有提到”推断对方未覆盖某个问题。正文采用主题综合，逐篇细节放在配套矩阵中，避免把 200 篇题名堆叠成未经分析的目录。

3 统一描述：修改哪里、凭什么、何时验证

3.1 干预对象与提案机制应分别标注

设模型输出为 f(x; θ, a)，其中 θ 为权重、a 为架构；第 l 层隐藏状态为 h_l。权重编辑改变 θ，激活引导改变特定输入上的 h_l，NAS 改变 a。提示改变 x，作为边界比较保留。每一种对象都可以接受统计构造、梯度训练、函数值搜索或语言模型生成的候选；对象相同并不意味着优化过程相同。[1, 51, 86, 131]



图 1 共同描述框架。横向是干预对象，纵向是提案与验证过程；连线表示可组合关系，不表示各分支已经等价或可以无损互换。

3.2 信息权限决定可用方法和公平基线

把可用信息写成 I = {输入输出、损失 / 奖励、权重、激活、梯度、模型代码、外部知识、环境状态}，比笼统的“黑盒”更清楚。BBT 在受限服务接口下搜索提示；MeZO 虽不反向传播，仍需要能扰动参数并得到目标值。[52, 58] 使用激活统计选择子空间的方法获得了额外内部信息；LLM 编辑器若能执行 Python、读取张量并批量测试，也比仅返回一个分数的查询接口丰富。[1, 85]

在这些权限之外，还要注明反馈数据是否带标签、能否重复访问测试集、可否调用更大的模型，以及候选是否允许额外推理程序。例如把求解过程写进 Python 控制流，与把该计算完全编译为固定网络参数，是不同的构造要求。[11, 50] “不更新主干参数” “不使用反向传播” “没有任务数据” “没有额外训练”是四个独立命题，不应相互替代。

3.3 局部关系有用，但不能推出全局等价

对于线性模块 y = Wx，在当前输入附近，权重变化产生 Δy = ΔW x。一个固定的激活偏移 v，要在多个输入上被同一 ΔW 实现，需要同时满足 ΔW x ≈ v；偏置、输入分布、秩和后续非线性都会影响可行性。单输入上存在一种参数实现，不保证它在长序列、其他任务或其他模型中保持同样语义。Steer2Edit 和表征式知识编辑为这种连接提供具体方法，但不构成任意方向均可安全写入权重的定理。[38, 99, 120]

同样，有限差分能对连续参数估计方向，却不能自动解决离散架构约束；NAS 的连续松弛需要额外的离散选择与验证。[75, 136, 156] 本文把跨空间关系视为需要检验的接口：记录输入条件、近似误差和失效范围，比给它们统一命名更有实际价值。

4 权重空间编程：从程序提案到可执行证据

4.1 阅读、构造与编辑是三个不同任务

WSP 中的“阅读”要求从参数推断网络行为；“构造”要求把目标计算写成网络；“编辑”则从已有模型出发改变行为。编辑成功可能利用模型已具备的能力或结构先验，不必然说明语言模型理解了全部参数的语义。反过来，能指出某个权重的局部作用，也不代表能构造稳定的全局能力。[1, 49, 50] Tracr 提供了从受限程序编译到 Transformer 的可控实例，有助于研究程序与参数的对应，但适用语言和结构边界应保留。[11]

典型闭环为：读取任务与模型信息，生成候选编辑程序，在反馈集上运行，接受或回退，保存与重新加载检查点，最后独立评估。这里既有候选的结构先验，也有反馈驱动的选择偏差。编辑器看到的题目、得分、日志、调用次数和失败候选，都是算法输入或成本的一部分。[1, 48]

来源	原文支持的结论	不能据此推出的结论
Exploring WSP [1] §2-3	所报告的 Qwen3.5-0.8B 数学轨迹从 8.35% 到 37.80%，修改 1,023 个参数；16 条轨迹中 6 条保留原权重	任意模型、任务和初始化均能得到这种增益
WeightLoop [48] §2.1	LLM 一次构造固定编辑器；实验反馈持续更新预测模型，并检验写回、重载和后续决策	编辑器本身在递归自我改写，或已完成在线湿实验验证
权重阅读审计 [49] §2.5-2.6 及实验	小型分类器上区分局部算术与数据集先验；标签置换暴露语义归因问题	对所有 WSP 编辑结果的直接复现或普遍否定
游戏难度阶梯 [50] 方法与验证章节	以独立求解器检验构造；局部战术、合法动作与长期对抗表现可以分离	有限游戏证明能推广到开放任务或一般智能

4.2 成本和失败必须按轨迹报告

[1] 的每次搜索预算是 50 次成功的前向工具请求；一次请求可含批量样本及其他信息，不能写成“只评估了 50 个样本”。8.35% 到 37.80% 是增加 29.45 个百分点，且是特定轨迹的结果。报告最佳检查点之外，还应给出所有轨迹的收益分布、回退次数、修改大小、提案模型费用与最终验证开销。稀疏编辑可能有高定位成本；参数数目也没有体现语言模型生成程序的计算。[1]

近期扩展的证据层级并不一致。图像复原方向的 WSP 方案明确把预期改善、诊断预算和递归循环写成技术猜想，而非已测得结果。[47] 本文将其作为待验证问题，不把其数字并入效果比较。原作者 Sparks 博客作为起始背景单独链接，未计入 200 篇学术文献，也不把其与后续论文不同协议下的成绩混合。[1, 47]

5 权重空间学习、生成与模型合并

5.1 参数数组不是天然对齐的语义表示

隐藏单元置换常能保持网络函数不变，却大幅改变参数排列。如果一个元模型把展平的参数当普通向量处理，它可能学到坐标偶然性。Permutation Equivariant Neural Functionals、等变权重对齐和 Universal Neural Functionals 分别处理置换结构、网络对齐及更一般的张量对称性。[12, 18, 22] UNF 对循环连接等结构给出构造，但其相关实验不能证明任意异构架构上的统一迁移；高阶交互的计算规模也是明确限制。[22]

权重空间数据增强试图减小模型库偏差；SANE、神经场适配和 WeightCLIP 则探索可伸缩表示或跨模态联系。[21, 28, 42, 45] 另一条路线是把网络放到一组输入上“询问”，用功能响应表示模型。ProbeGen 的结果提醒我们：读取权重的方法应与有相同查询预算的功能探针比较，而非只和简单参数统计比较。[33] 二者的信息成本和适用架构不同，不能仅凭“直接读参数”判断优劣。

5.2 生成网络需要把训练资产计入账本

HyperNetworks 学习从条件生成另一网络的权重；压缩权重表示和 HyperDiffusion 在低维表示或训练过的模型集合上学习分布。[2, 5, 14] 后续工作将这一路线用于定制扩散模型的理解、形状生成、机器人适配或跨架构映射。[27, 39, 44, 46] 这些方法的快速生成可能依赖大量已有模型、专家适配器或配对样本。它们与“不为当前任务训练专用权重生成器”的通用 LLM 提案有联系，但不能省略预训练这一关键区别。

方法家族	关键操作与条件	成本与主要局限
低秩适配 [3, 13, 15, 23]	训练低秩增量；自适应秩、量化基座或幅值 / 方向分解	仍有训练和梯度成本；低可训练参数量不等于低总显存
Model soups / 任务算术 [7, 10]	组合兼容检查点或相对基座的任务向量	来源模型训练已付费；坐标、初始化和任务冲突影响组合
Git Re-Basin / 对齐 [8, 18]	通过置换等结构寻找可组合表示	对齐本身有成本；不同架构与复杂对称性增加困难
TIES、DARE、稀疏与信赖域合并 [17, 20, 31, 35]	修剪、符号协调、增量稀疏化或限制合并范围	需要验证集或选择规则；压缩后的负迁移仍需检测
超网络及元学习 [2, 14, 44, 46]	训练生成器或映射器，之后快速给出权重	训练数据、模型库及新架构迁移共同限制收益

5.3 合并、生成与理解之间的推理边界

模型合并能保留若干能力，不意味着任务向量对应一个唯一、可解释的概念；权重生成能产生有效模型，也不意味着生成器逐项理解参数。比较应同时记录基座同源性、结构兼容性、目标任务是否见过、源检查点训练预算与部署时是否真正只保留单个模型。[7, 8, 10, 17, 20, 43] 对 WSP 而言，这些方法提供有价值的候选生成与约束工具；是否降低总成本，还需要把整个流程放回同一任务和预算下检验。

6 知识编辑：局部修补与一致性之间

6.1 不同编辑器依赖不同先验

ROME 将事实关联定位到特定参数结构后进行更新，MEMIT 扩展到批量记忆编辑，MEND 则训练编辑网络处理更新信息。[4, 6, 9] 它们都可能产生局部参数修改，却不因此属于同一种优化器。前两类的定位、表示优化和统计量准备，与训练元编辑器的成本应分别记录。SAKE 从事实分布与激活角度组织知识更新，Steer2Edit 则把引导相关方向连接到权重操作，进一步说明“编辑对象”与“提案机制”需要多标签分类。[38, 120]

对定位后编辑的方法，准备阶段可能主导首次使用时间。Minimal Precomputation 在所测单次和批量编辑设定中，发现可以显著减少所需隐藏向量统计。[40] 该文也明确未解决顺序编辑下统计量需求及下游能力损失的关系。因此“单次更新很快”与“第一次为新模型完成编辑很快”是不同结论；跨模型部署和持续维护需要单独预算。

6.2 编辑成功率不足以衡量知识改变

MQuAKE 用多跳问题检查事实更新的连带后果：能正确复述被编辑事实，并不保证推理结果同步改变。[16] AKEW 的自然文本更新、事件级编辑和跨语言多跳编辑进一步扩大了输入形式及泛化要求。[25, 26, 30] HalluEditBench 提醒我们应先确认原模型确实答错，再评价编辑是否修复幻觉，否则“编辑成功”可能只是在测试本来已知的内容。[34]

评价维度	应区分的测试对象	容易被隐藏的问题
目标可靠性	编辑模板、释义及不同上下文	只记住模板或关键词
局部性	无关事实、相邻关系、一般能力	过强局部性约束也可能阻止合理连带改变
一致性	多跳后果、事件参与者、跨语言表达	单条事实正确但关联答案仍旧
持续编辑	多次更新、冲突更新、撤销与回滚	参数漂移、注意力异常和累计能力回退
真实使用	原本错误的事实、开放提问及分布外输入	基准成功率与真实修复率混淆

6.3 连续编辑与协作更新暴露新的冲突

稳定知识编辑强调上下文变化；终身编辑和过度注意研究关注重复更新引发的内部失衡。[24, 36, 37] CollabEdit 与面向编辑的模型合并考虑多个编辑来源之间的协调，但分布式更新本身不会消除冲突和版本管理需求。[32, 41] 用编辑分析电路，也需要区分干预造成的行为变化与原模型本来采用的计算过程。[29]

据此，本文把“修补”看作带有保留约束的更新问题：目标事实要改变，合理后果应传播，无关能力应保留，冲突应可追踪。不存在一个单一成功率能同时表示这四个要求。[19] 对 WSP 的启示是，编辑程序必须附带可执行的目标、保留和回退测试；只展示新答案或稀疏补丁，不能构成完整的可靠性证据。

7 零阶优化：函数值如何形成更新方向

7.1 基本估计器与查询单位

零阶优化通过目标函数值构造搜索方向。以参数 θ、批次 B、随机方向 u、扰动尺度 ε 为例，单方向双边估计可写为下式。这里 u 采用标准高斯等满足相应条件的采样；若使用单位球方向，归一化系数需随定义调整。[51, 55, 56]

$$\hat{g} = \{L(\theta + \varepsilon u; B) - L(\theta - \varepsilon u; B)\} u / (2\varepsilon), \quad \theta' = \theta - \eta \hat{g}。$$

在基本设定中，每个方向需要两次批次目标评估；使用 q 个方向约需 2q 次。这不是每次更新只看两个训练样本，也不是总训练只需两次推理。对于生成式任务，单次评估还可能包括多个输出 token、采样、奖励计算或完整轨迹。MeZO 利用随机种子重建扰动，避免像普通反向传播那样保留大量中间激活，但仍要执行扰动、前向和参数写入。[52]

Natural Evolution Strategies 与大规模 ES 从参数分布及适应度估计更新方向，提供另一种函数值搜索视角。[53, 54] 其并行种群、通信方式与任务地平线不同于单方向微调；当损失不连续、奖励稀疏或维度很高时，理论假设和实践方差都值得单独检查。一般无导数优化还包括局部模型、直接搜索等，不宜被“有限差分”完全代表。[55, 56]

7.2 降低有效维度与降低方差

路线及代表文献	主要改变	收益的代价或边界
稀疏扰动与掩码 [57, 62, 73]	仅扰动部分坐标，或按任务挑选参数	掩码构建、更新频率及漏掉有效方向的风险
低秩 / 子空间 [66, 68, 69, 84, 85]	在张量分解、随机或学习到的子空间搜索	子空间外能力难以修正；构建可能需要激活或训练
方差减少与归一化 [65, 78, 79]	复用估计、对损失差做尺度调整	附加查询、参考状态及适用条件需披露
分层和混合更新 [61, 67, 70, 71]	层选择、投影或一阶 / 零阶结合	一阶部分需要梯度；不能统一标为纯无梯度
低精度与量化 [74]	扰动和更新适配量化计算	舍入噪声可能淹没小扰动；需联合选择尺度

减少搜索维度并不等于免费减少难度。随机子空间降低估计负担，同时限制可达更新；学习子空间引入更强先验和准备工作。MaZO 关注多任务参数冲突，DiZO 关注层间更新差异，StructZO 使用结构信息，优化的瓶颈并不完全相同。[71, 73, 85] 因而同为“ZO”，也应在相同任务、可用信息和达到同等效果的条件下比较，而不是只按一次迭代的前向次数排序。

8 零阶优化：显存、时间与服务接口的权衡

8.1 内存优势不能直接换成训练速度优势

ZO 通常减少反向图与激活存储，但总时间还受扰动生成、参数读写、批量调度和迭代数影响。ZO-LLM Benchmark 比较多种模型、任务复杂度和微调方案，表明任务与预训练形式的对齐会影响表现，也讨论块更新和一阶 / 零阶混合。[61] 这解释了为何一次短任务上的显存收益，不能直接预测长推理、低信噪奖励或不同目标函数上的收敛速度。

FZOO 使用共享基点的一侧估计、多方向扰动和损失变化归一化，并结合 Rademacher 方向与执行优化。[78] 若同一批次中 q 个方向共享基点，目标评估为 q + 1 次；它不能被概括成“每个方向都只要一次且无基点”。Inference-Native ZO 则把优化与推理系统的执行路径相联系。[81] 两者提示我们，算法查询数与硬件吞吐是不同层面的指标：同样的理论次数，因批处理、缓存和内核实现可能呈现不同墙钟时间。

8.2 黑盒适配、隐私与通信并非同一个问题

BBT、局部提示搜索与 ZIP 在语言或视觉语言服务接口下调整连续提示，适用的可写空间与全参数 ZO 不同。[58, 64, 76] 联邦 ZO 研究需同时报告通信轮次、客户端异构性、本地查询及收敛条件；差分隐私方法还要明确噪声机制和隐私预算。[59, 60, 72] “不发送梯度”本身既不提供完整隐私证明，也不能代表通信成本已降低到可忽略。

EvoLLM 和 AdaEvolve 使用语言模型改写或组合候选，把程序 / 提案搜索与函数值反馈连接起来。[63, 80] ZO-DARTS++ 将零阶估计引入架构优化；GRZO 则关注组相对反馈。[75, 82] 这些交叉点适合与 WSP 对照，但其候选空间、反馈粒度和失败约束必须先对齐。不能因为都“用分数改候选”，就跳过模型访问权限和计算单元的差异。

比较目标	建议固定的条件	必须同时报告
显存节约	同一基座、序列长度、批量和精度	峰值显存、可训练参数、临时缓冲区
时间效率	同任务、硬件及目标质量阈值	准备、每步耗时、步数、失败重启
查询效率	同评估样本、生成长度和反馈信息	函数调用、样本数、token 数或轨迹数
系统可用性	同服务权限和并行资源	参数写入、缓存占用、通信及部署限制

8.3 应用与理论的适用边界

关于平坦极小值和自适应 ZO 的分析，为理解扰动与泛化提供了假设明确的视角，不应据此断言所有 LLM 微调都更稳健。[77, 79] VLA-ZO 通过动作分支适配及视觉语言前缀复用寻求效率，体现的是结构与缓存共同带来的收益；迁移到不同机器人、动作表示或动态环境后仍需重新测量。[83] 实践上应同时绘制“质量-查询”“质量-时间”“质量-显存”三类曲线，并公布达到失败或停机条件的运行，才看得见可用预算下的真实权衡。

9 表征引导：方向、位置、强度与时机

9.1 从统计方向到可训练干预

常见激活引导在第 l 层施加 h'_l = h_l + αv。v 是方向，α 是强度；这一简式隐藏了方向如何估计、在哪些 token 上施加、是否按上下文门控等选择。早期潜在引导向量、ActAdd、ITI、RepE 和 CAA 分别从目标句、对比输入、注意力头或行为表征出发构造干预。[89-95] PPLM 在生成过程中使用梯度调整隐状态，其在线代价与直接向量相加不同；ReFT 则训练低维表征变换，不能只用“参数不更新”描述其准备过程。[88, 99]

对比样本均值之差可以提供低成本方向，但模板、长度或无关语境也可能进入向量。均值中心化研究针对这种估计偏差；面向广泛技能与多行为的工作表明，方向组合和行为范围需要单独验证。[94, 98] 指令遵循、视觉语言幻觉与偏好引导分别提供特定任务下的结果，不应被汇总为一个跨任务的通用可控性分数。[103, 104, 109]

方法家族	方向 / 策略来源	准备与运行成本	主要限制
对比式方向 [91, 95]	成对样本的激活统计	激活采集；每层 / token 注入	对比集偏差、层与强度敏感
有监督定位或训练 [90, 99]	标签、探针或低维优化	训练与验证；在线投影	权限、标签与任务迁移
SAE 方向 [92, 100, 108]	稀疏分解及特征选择	训练字典、标注 / 选特征	重构好不代表干预因果有效
条件 / 动态控制 [102, 113, 114]	上下文门控或反馈规则	门控校准、运行时状态	误触发、控制信号与目标错位
超网络方向 [110]	由条件生成引导向量	辅助模型训练；生成和注入	白盒访问及训练分布依赖

9.2 门控让干预更细，但也增加自由度

CAST 依据条件触发引导，动态缩放与深度方向优化调节何处、何时和多强地施加干预，PID 式方法尝试按反馈调整。[102, 113-115] 更细粒度的 AUSteer、上下文线性方法、核化方向和因果中介生成分别引入新的选择或建模方式。[118, 121, 122, 128] 每增加一种控制自由度，也增加标定成本与过拟合验证集的机会，应报告调参预算而非只报告最终向量。

跨语言引导与上下文线性引导在文献中都使用 CLAS 缩写，但任务、构造及证据不同，应按完整题名区分。[117, 122] 这种命名重叠说明，仅按方法名称归类容易误读。本文更关注四个可复核字段：方向估计、注入位置、系数 / 门控策略、验证分布；它们也是复现一个引导结果时最少需要记录的信息。[86, 107]

10 表征引导：可解释性与副作用的证据

10.1 稀疏特征的可读性不等于控制可靠性

SAE 把激活表示为稀疏特征组合；扩展规模、JumpReLU、BatchTopK 和 Transcoder 等路线分别调整稀疏机制或学习目标。[92, 100, 101, 105, 106] 对字典的评价至少包括重构、稀疏性、特征可解释性、敏感性和因果干预效果。它们并非互相保证：一个人能给特征命名，不代表该特征对所有含义相近的样本都响应；更宽字典也不必然得到更好的行为控制。[112, 125]

SAEBench 可靠性审计通过重复评估、合成可知真值和不同质量面板检查指标，发现部分指标存在噪声、区分度或效度问题。[124] 该结论有具体模型和采样限制，不能被改写成“SAE 都无效”。它实际要求研究者说明：同一字典换种子后分数会变多少，变化来自特征质量还是评估抽样，以及基准指标是否对应目标行为。

10.2 定位与干预验证应保持两步

AtP* 用高效近似筛选可能重要的节点，再对候选执行更昂贵的干预验证，并诊断漏检。[97] 这条路线适合把可解释性作为搜索的提案器，但它并未消除真实干预的必要性。用梯度近似定位、用相关性选特征和用消融证实因果，证据强度各不相同；基线激活的选择、提示对分布和节点粒度也会改变结论。对于 WSP，类似原则是让解释提出补丁，再让独立行为测试决定接受，而不以解释的流畅程度代替正确性。[116]

10.3 效果应与副作用共同报告

SteerScope 在所测两种 Gemma-2 规模、23 种方法和 15 项指标框架下比较目标效果与副作用；默认聚合中，没有被评估的激活方法同时支配提示引导。[87] 这不表示提示在每个子任务和任意预算下都更好，也不否定其他模型上训练辅助网络的收益。HyperSteer 在其训练和评估设定中展示对未见引导提示的迁移，但有额外训练、白盒访问及生成 / 评估数据的成本。[110] 两类结果的比较对象不同，不能简单判为互相矛盾。

对齐与有用性权衡、安全机制变化、内生抵抗和曲率分析分别指出引导可能触及模型已有决策结构。[96, 111, 119, 129] 副作用预测研究还发现行为之间的交互可不对称；余弦相似度却是对称量，因此仅靠方向夹角无法表达全部交互。[126] 最小化附带损害的约束提供另一种目标，但需要验证所约束的统计量是否对应实际保留能力。[123]

语义特异性研究提示，某些看似推理改善的收益可能来自标签偏置；减少超参数扫描的方向几何指标，在开放生成上也未必保持同等预测能力。[127, 130] 因而评价应包含换标签、换规则、换模板、自由生成和保留任务，并呈现不同强度下的收益-损害曲线。若只挑一个使目标得分最高的系数，控制效果往往被高估。

11 NAS：搜索策略、性能估计和最终架构

11.1 三个部件决定一个 NAS 系统

NAS 需要给定搜索空间、候选提案 / 搜索策略以及性能估计方式。[131, 143] 强化学习搜索和正则化演化通过训练后的适应度指导架构选择；Progressive NAS 以逐步扩展和预测降低评估负担。[132-134] ENAS 共享参数，Graph HyperNetworks 预测候选权重，两者都把每个候选独立训练的开销，转移到一个可复用的训练资产上。[135, 137] 这与权重生成和模型库研究相通，但候选评估偏差也随之进入选择过程。

可微方法把操作选择松弛为连续参数，交替优化网络权重与架构参数。DARTS、SNAS 及其后续方法分别采用确定性或随机的松弛；PC-DARTS、P-DARTS、DrNAS 又改变通道、深度或架构分布。[136, 139, 142, 144, 151] 连续解最终仍需变成离散架构，训练和选择之间的差距不能靠可微形式自动消除。

路线	代表方法	主要成本	典型评估风险
独立候选评估	RL、演化、渐进搜索 [132-134]	多候选训练与调度	少量运行、昂贵基准和极值选择
共享 / 预测权重	ENAS、GHN、OFA [135, 137, 145]	超网或权重预测器训练	共享权重排名不等于独立训练排名
可微搜索	DARTS、SNAS、DrNAS [136, 139, 151]	双层优化及离散化	松弛偏差、跳连占优、选择规则
低成本代理	NASWOT、zero-cost、KNAS [149, 154, 158]	少量批次、梯度或统计量	代理随空间改变；仍需最终训练
LLM / 生成式搜索	EvoPrompting、LLMatic、DIFFNAS [160, 162, 175]	提案、代码验证、训练或生成器	先验、历史资产和失败代码未计费

11.2 硬件目标与最终训练不能被搜索分数替代

ProxylessNAS 将目标硬件上的资源约束纳入设计；Once-for-All 先训练可覆盖多个配置的超网，再为设备选择子网络。[138, 145] 这里的摊销价值取决于部署设备数，而不只是单次查询速度。多目标 NAS 引入精度、延迟或其他偏好；碳效率研究还依赖电力和调度假设。[163, 164] FLOPs、参数量、能耗和实测延迟不是可互换的单位。

以 DARTS 原文 CIFAR-10 比较为例，一阶 / 二阶搜索成本报告为 1.5 / 4 GPU 天，但相关比较表说明另有候选选择与最终训练成本。[136] 因而引用这组数字时应标明“搜索阶段”，不能称为整个方法的总成本；它也不适合作为跨年份、跨硬件的通用速度基准。真正的部署比较必须在固定训练配方和硬件下重新评估选出的离散架构。

12 NAS：可靠比较、语言模型先验与迁移

12.1 随机基线与训练配方是必要对照

Random Search and Reproducibility 显示随机搜索结合早停或权重共享可以成为有竞争力的基线，也区分“能否重现已报告运行”与“结论是否对随机性稳健”。[140] NAS 最佳实践进一步要求公开空间、训练配方、种子与搜索成本。[146] 若改进来自更好的超网训练，而只归功于搜索器，结论就会混淆；对超网启发式的分析正是在检查这一问题。[157]

DARTS 的架构权重大小未必对应操作的真实贡献，扰动式选择通过移除操作观察性能变化，重新检验离散化规则。[156] 稳健 DARTS 与邻域感知方法分别从曲率和局部稳定性处理搜索偏差。[147, 155] 因此“搜索器找到高分结构”之后，还要问：它在另一初始化、独立训练和轻微空间改变后是否仍然优于随机选择？

12.2 表格、代理与零成本指标各有适用域

NAS-Bench-101、201 和 NATS-Bench 提供可重复比较的有限空间；NAS-Bench-NLP 扩展任务类型，NAS-Bench-301 以代理模型覆盖更大的空间。[141, 148, 150, 152, 153] 查询表格能降低研究者重复训练的成本，但造表成本并未消失；在固定表格上优化，也不能代表对真实硬件、任意架构或新任务的泛化。代理基准还应计入模型误差，防止搜索器专门利用预测器弱点。

“零成本”代理通常指避免完整训练，而非零计算、零数据或零梯度。NASWOT、梯度相关代理、KNAS、LiteTransformerSearch 和 ZeroLM 使用不同信号，比较时应包括简单参数量 / 宽深规则等基线。[149, 154, 158, 159, 170] 如果某个空间里参数量已能解释多数性能差异，复杂代理的优势必须在控制这种结构因素后确认。

12.3 大模型与历史搜索改变了成本起点

EvoPrompting、LLMatic、设计原则迁移、SEKI 与 Agentic NAS 让语言模型生成代码、总结候选或约束后续搜索。[160, 162, 167, 169, 172] 这些方法的先验可能来自预训练中的公开架构，需把新结构探索与已有设计复用区分。DIFFNAS 引入生成模型；QuaNAS 与稀疏历史搜索利用既有搜索资产。[173-175] 对它们公平的成本比较应同时给出“资产已经存在”和“从零建立资产”两个情景。

在语言模型应用中，NAS 又与参数高效微调、结构剪枝、低秩适配和弹性架构结合。[161, 165, 166, 168, 171] 目标可能是压缩、设备适配或多任务选择，不能统一用分类精度衡量。与 WSP 的共同点在于候选程序需接受执行验证；差别在于 NAS 常改变计算结构并触发重训，而 WSP 的某些设定要求在既定架构内直接写入参数。把这项约束写清楚，才能判断两者是否共享一个合理基线。

13 具身控制：干预进入真实时间以后

13.1 动作表示决定可操作接口

RT-1、RT-2、Open X-Embodiment、Octo、OpenVLA、Diffusion Policy 与 pi_0 分别体现动作 token、跨机器人数据、通用策略以及扩散 / 流式动作生成等路线。[178, 179, 181-185] 这些基础模型的离线训练是后续适配的前提。即使目标任务不重新训练主干，也不能把已有策略能力和数据资产算作免费。RLBench 与 LIBERO 为操作学习提供评估环境，但环境内成功率和真实部署可靠性仍是不同指标。[177, 180]

干预接口	代表研究	需计入的额外过程	主要验证问题
潜在动作表征	SteerAct、VLA 机制引导 [176, 189]	SAE / 语义定位、参考轨迹与门控	方向语义、干预时机与迁移
外部视觉反馈	VLS [192]	视觉语言评估、观测与策略接口	反馈延迟及判断错误的影响
动作 token	token intervention [196]	token 选择、干预强度或约束	离散动作的物理可行性
采样 / 流过程	Flow Control [195]	采样调节及交互信息	多步生成成本、控制延迟与稳定性
训练或参数适配	RIPT-VLA、VLA-ZO [187, 83]	交互轨迹、奖励或参数查询	与推理干预匹配数据预算

13.2 时间门控不只是“加一个方向”

SteerAct 利用少量有结果标签的参考轨迹校准时序引导；默认仿真设置为每任务 24 条参考 rollout，并依赖预训练 SAE。[176] 这不是 24 条全成功示范，也不表示完全没有准备数据。方向、触发条件和注入时机共同影响行为。原文的时机 / 延迟消融有启发性，但不能单独证明某一个阶段变量具有唯一因果作用。VLA 机制引导则提示，相近语义词可能对应“慢而谨慎”或“慢而卡住”等不同动作后果。[189]

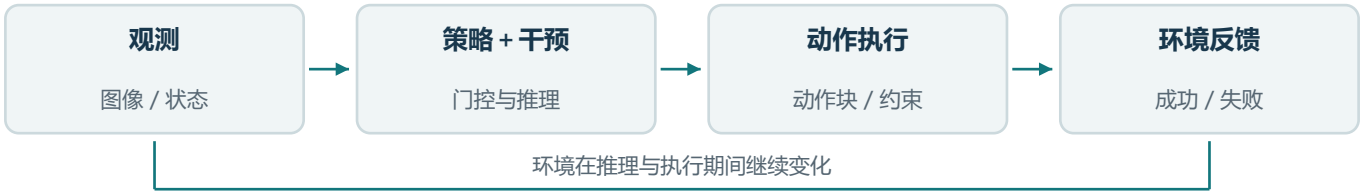


图 2 机器人干预的闭环。推理延迟、动作分块与环境变化共同决定下一次可用反馈；离线单步得分遗漏了这个时间过程。

13.3 成功率之外的失效维度

安全约束、置信度校准、动作幻觉和变形测试分别检查可行性、可置信程度及输入变化下的稳定性。[186, 188, 193, 194] 具身泛化、跨硬件差距和效率综述说明，任务迁移不等于接口或本体迁移。[190, 191, 197] LIBERO-RECOVER、VPro 和 Dynamic 进一步关注失败恢复、陈旧视觉及随推理时间推进的环境。[198-200] 表征引导不能替代动作约束和环境测试；应同时报告恢复、碰撞、延迟和长任务完成情况。

14 横向综合：把方法放回同一本成本账

14.1 四阶段记录，而不是一个“效率”数字

不同方法把成本放在不同阶段。低秩编辑节省可训练参数，ZO 节省反向激活，权重共享节省重复候选训练，超网络把逐任务优化转移到辅助模型训练，推理引导则把部分成本转到每次部署。[2, 3, 52, 99, 110, 135, 145] 用单一 GPU 天、参数个数或工具调用数混合排序，会掩盖这些差异。本文建议如下账本；它是综合分析框架，不是新增实验结果。

阶段	应记录的项目	典型遗漏
准备 C_pre	基座资产、模型库、统计量、SAE / 超网络、参考轨迹、编译	只计目标任务增量，把辅助训练称为免费
搜索 C_search	提案、方向数、样本 / token / 轨迹、候选训练、失败重试	只报最佳运行，忽略调参与无效候选
部署 C_deploy	每次推理的注入、额外前向、缓存、门控和端到端延迟	用离线模型 FLOPs 代替真实控制时间
验证 C_eval	独立保留集、随机种子、能力回退、OOD 和闭环实验	把参与选择的验证集当最终测试集

$$C_{total}(K) = C_{pre} + C_{search} + K \cdot C_{deploy} + C_{eval}。$$

只有在单位一致时，上式才能数值相加；实际账本应为时间、GPU 时间、峰值显存、通信、人工和环境交互的多维记录。K 表示部署次数。如果两种方法的准备成本与单次成本不同，可在相同质量和资源条件下计算摊销转折点；若一次完成后适用于很多任务，辅助训练可能值得，若只处理一次性任务则可能不值得。本文没有给出通用转折点，因为现有文献的任务和硬件不足以支持统一数字。

14.2 三类常见“免费”错觉

第一是资产已付费：模型 soups、历史 NAS 与超网络把既有检查点或生成器作为输入。[7, 110, 173] 第二是接口信息被省略：激活选择的子空间与仅函数值的随机方向并非同等权限。[69, 85] 第三是成功后的维护被省略：知识编辑的顺序漂移、机器人失败恢复和引导的副作用会产生持续验证成本。[36, 87, 198] 因此报告应同时给出首次使用和已有资产情况下的增量成本，并列出资产何时需要重新生成。

14.3 公平比较的最小单元

一项可复核的比较至少固定基座、任务分布、信息权限、反馈数据、搜索预算和最终测试；记录多个随机种子，并保留完整失败轨迹。效果维度包括目标收益、保留能力、迁移、稳定性和运行约束。对于不能对齐的论文，应并列说明设计差异，而不是强行排出名次。跨空间研究最有价值的结果往往不是“谁普遍更好”，而是“在什么权限、预算和部署频率下，哪种干预值得尝试”。

15 可检验的研究问题与结论边界

15.1 从宏大能力表述转为可证伪实验

问题	建议实验及必要对照	可以支持的有限结论
LLM 是否利用了权重语义?	对网络置换、标签置换、函数保持变换和陌生数据集做盲测；控制代码 / 工具权限 [22, 49]	区分局部计算、数据先验与函数层面的理解
解释能否提高编辑效率?	同预算比较解释提案、随机提案和 ZO；由独立测试接受补丁 [1, 52, 97]	定位信息在特定空间中的增量价值
引导能否稳定写入权重?	比较输入内拟合、跨输入、跨任务及长序列；报告保留能力 [38, 120]	激活到参数转换的有效分布与误差
搜索收益来自算法还是资产?	固定训练配方，加入随机搜索；分别有 / 无模型库和历史记录 [140, 157, 173]	搜索器、先验和训练资产各自的贡献
机器人时机是否具有因果作用?	匹配干预幅度与总次数，随机化触发时刻，控制推理延迟 [176, 198-200]	特定任务中时机而非剂量带来的收益

这些是后续研究建议，尚未由本项目实施。实验应预先给出停机条件和失败定义，例如收益未超过种子波动、保留能力低于阈值、出现不可恢复状态或超出资源预算。对参数编辑，保留原检查点及可逆补丁；对在线控制，采用环境级约束和回退策略。这里的目的不是把每项研究都变成同一大基准，而是选择最能排除替代解释的最小实验。

15.2 证据仍然存在的空缺

当前直接 WSP 证据相对集中于少数模型、程序构造与特定任务；较新预印本的结果还需要独立复核。激活引导的方向和系数可塑性很强，但评价可能受模板、标签和裁判模型影响。ZO 的显存收益较明确，任务匹配、估计方差和实际系统吞吐仍共同决定可用性。NAS 的可复现基准成熟度较高，却受搜索空间、训练配方和代理误差限制。具身控制最接近部署后果，也最难统一环境状态、时间和硬件条件。[1, 49, 61, 87, 124, 146, 197]

本综述的 200 篇样本采用分支覆盖和代表性选择，受查询词、返回上限及 arXiv 收录偏差影响；未进行双人独立筛选、质量量表评分或元分析。62 篇“全文相关章节核验”不等于全部逐页精读，138 篇摘要筛选也不足以支撑细粒度性能复述。引用较新论文是为了定位研究方向，并非对全部结论背书。上述边界随公开文献矩阵保留，后续更新可以继续提高证据深度。

15.3 结论

权重空间编程的研究价值，在于把参数结构、可执行提案与反馈验证联系起来。邻近领域已提供许多可以借鉴的部件：等变权重表示约束坐标问题，知识编辑揭示局部性与一致性，ZO 提供函数值更新机制，表征研究提供定位和控制接口，NAS 提供搜索与代理评估经验，具身控制则迫使这些方法接受时间和物理后果的检验。可靠的下一步，是在明确的信息权限和全流程预算内证明一项具体改进，同时公开失败与迁移边界，而不是从少量漂亮补丁推导通用能力。

参考文献按主组排列，编号与正文及配套矩阵一致。题名链接指向原论文；版本和核验记录以配套元数据及证据表为准。原始博客仅作补充背景，见仓库 README。

参考文献 1-20

[1] Junru Hu et al. (2026). Exploring Weight-Space Programming: Capabilities, Generalization, and Limitations. arXiv:2611.09382.

[2] David Ha et al. (2016). HyperNetworks. arXiv:1609.09106.

[3] Edward J. Hu et al. (2021). LoRA: Low-Rank Adaptation of Large Language Models. arXiv:2106.09685.

[4] Eric Mitchell et al. (2021). Fast Model Editing at Scale. arXiv:2110.11309.

[5] Kazuki Irie et al. (2021). Training and Generating Neural Networks in Compressed Weight Space. arXiv:2112.15545.

[6] Kevin Meng et al. (2022). Locating and Editing Factual Associations in GPT. arXiv:2202.05262.

[7] Mitchell Wortsman et al. (2022). Model soups: averaging weights of multiple fine-tuned models improves accuracy without increasing inference time. arXiv:2203.05482.

[8] Samuel K. Ainsworth et al. (2022). Git Re-Basin: Merging Models modulo Permutation Symmetries. arXiv:2209.04836.

[9] Kevin Meng et al. (2022). Mass-Editing Memory in a Transformer. arXiv:2210.07229.

[10] Gabriel Ilharco et al. (2022). Editing Models with Task Arithmetic. arXiv:2212.04089.

[11] David Lindner et al. (2023). Tracr: Compiled Transformers as a Laboratory for Interpretability. arXiv:2301.05062.

[12] Allan Zhou et al. (2023). Permutation Equivariant Neural Functionals. arXiv:2302.14040.

[13] Qingru Zhang et al. (2023). AdaLoRA: Adaptive Budget Allocation for Parameter-Efficient Fine-Tuning. arXiv:2303.10512.

[14] Ziya Erkoç et al. (2023). HyperDiffusion: Generating Implicit Neural Fields with Weight-Space Diffusion. arXiv:2303.17015.

[15] Tim Dettmers et al. (2023). QLoRA: Efficient Finetuning of Quantized LLMs. arXiv:2305.14314.

[16] Zexuan Zhong et al. (2023). MQuAKE: Assessing Knowledge Editing in Language Models via Multi-Hop Questions. arXiv:2305.14795.

[17] Prateek Yadav et al. (2023). TIES-Merging: Resolving Interference When Merging Models. arXiv:2306.01708.

[18] Aviv Navon et al. (2023). Equivariant Deep Weight Space Alignment. arXiv:2310.13397.

[19] Vittorio Mazzia et al. (2023). A Survey on Knowledge Editing of Neural Networks. arXiv:2310.19704.

[20] Le Yu et al. (2023). Language Models are Super Mario: Absorbing Abilities from Homologous Models as a Free Lunch. arXiv:2311.03099.

参考文献 21-40

[21] Aviv Shamsian et al. (2024). Improved Generalization of Weight Space Networks via Augmentations. arXiv:2402.04081.

[22] Allan Zhou et al. (2024). Universal Neural Functionals. arXiv:2402.05232.

[23] Shih-Yang Liu et al. (2024). DoRA: Weight-Decomposed Low-Rank Adaptation. arXiv:2402.09353.

[24] Zihao Wei et al. (2024). Stable Knowledge Editing in Large Language Models. arXiv:2402.13048.

[25] Hao Peng et al. (2024). Event-level Knowledge Editing. arXiv:2402.13093.

[26] Xiaobao Wu et al. (2024). AKEW: Assessing Knowledge Editing in the Wild. arXiv:2402.18909.

[27] Amil Dravid et al. (2024). Interpreting the Weight Space of Customized Diffusion Models. arXiv:2406.09413.

[28] Konstantin Schürholt et al. (2024). Towards Scalable and Versatile Weight Space Learning. arXiv:2406.09997.

[29] Huaizhi Ge et al. (2024). Understanding Language Model Circuits through Knowledge Editing. arXiv:2406.17241.

[30] Aditi Khandelwal et al. (2024). Cross-Lingual Multi-Hop Knowledge Editing. arXiv:2407.10275.

[31] Yifei He et al. (2024). Localize-and-Stitch: Efficient Model Merging via Sparse Task Arithmetic. arXiv:2408.13656.

[32] Jiamu Zheng et al. (2024). CollabEdit: Towards Non-destructive Collaborative Knowledge Editing. arXiv:2410.09508.

[33] Jonathan Kahana et al. (2024). Deep Linear Probe Generators for Weight Space Learning. arXiv:2410.10811.

[34] Baixiang Huang et al. (2024). Can Knowledge Editing Really Correct Hallucinations?. arXiv:2410.16251.

[35] Wenju Sun et al. (2025). Task Arithmetic in Trust Region: A Training-Free Model Merging Approach to Navigate Knowledge Conflicts. arXiv:2501.15065.

[36] Akshat Gupta et al. (2025). Lifelong Knowledge Editing requires Better Regularization. arXiv:2502.01636.

[37] Pinzheng Wang et al. (2025). Revealing and Mitigating Over-Attention in Knowledge Editing. arXiv:2502.14838.

[38] Marco Scialanga et al. (2025). SAKE: Steering Activations for Knowledge Editing. arXiv:2503.01751.

[39] Maximilian Plattner et al. (2025). Shape Generation via Weight Space Learning. arXiv:2503.21830.

[40] Akshat Gupta et al. (2025). Efficient Knowledge Editing via Minimal Precomputation. arXiv:2506.04226.

参考文献 41-60

[41] Zichuan Fu et al. (2025). Model Merging for Knowledge Editing. arXiv:2506.12384.

[42] Zhuoqian Yang et al. (2025). Weight Space Representation Learning via Neural Field Adaptation. arXiv:2512.01759.

[43] Xiaolong Han et al. (2026). A Survey of Weight Space Learning: Understanding, Representation, and Generation. arXiv:2603.10090.

[44] Christian Bianchi et al. (2026). Robotic Policy Adaptation via Weight-Space Meta-Learning. arXiv:2606.07217.

[45] Aron Asefaw et al. (2026). WeightCLIP: Aligning Datasets and Models for Weight Space Learning. arXiv:2607.03551.

[46] Adir Dayan et al. (2026). CrossGMN: Graph Metanetworks for Cross-Architecture Weight-Space Transformations. arXiv:2610.01649.

[47] Zhuoran Zheng et al. (2026). Weight-Space Programming for Improving Image-Restoration Diffusion Models: Technical Conjectures and a Recursive Improvement Loop. arXiv:2611.04268.

[48] Mengran Li et al. (2026). WeightLoop: Weight-Space Programming with Experimental Feedback. arXiv:2611.07319.

[49] Nazmus Sakib Ahmed (2026). Can an LLM Actually Reason and Program about Neural Network's Weights or is it just memorization on steroids? Exploring "Weight-Space Programming". arXiv:2611.08070.

[50] Eduardo C. Garrido-Merchán (2026). Zero-Shot Weight-Space Programming Across a Difficulty-Graded Ladder of Two-Player Games. arXiv:2611.09271.

[51] Sijia Liu et al. (2020). A Primer on Zeroth-Order Optimization in Signal Processing and Machine Learning. arXiv:2006.06224.

[52] Sadhika Malladi et al. (2023). Fine-Tuning Language Models with Just Forward Passes. arXiv:2305.17333.

[53] Daan Wierstra et al. (2011). Natural Evolution Strategies. arXiv:1106.4487.

[54] Tim Salimans et al. (2017). Evolution Strategies as a Scalable Alternative to Reinforcement Learning. arXiv:1703.03864.

[55] Krishnakumar Balasubramanian et al. (2018). Zeroth-order Nonconvex Stochastic Optimization: Handling Constraints, High-Dimensionality and Saddle-Points. arXiv:1809.06474.

[56] Jeffrey Larson et al. (2019). Derivative-free optimization methods. arXiv:1904.11585.

[57] Mayumi Ohta et al. (2020). Sparse Perturbations for Improved Convergence in Stochastic Zeroth-Order Optimization. arXiv:2006.01759.

[58] Tianxiang Sun et al. (2022). Black-Box Tuning for Language-Model-as-a-Service. arXiv:2201.03514.

[59] Zhenqing Ling et al. (2024). On the Convergence of Zeroth-Order Federated Tuning for Large Language Models. arXiv:2402.05926.

[60] Z Liu et al. (2024). Differentially Private Zeroth-Order Methods for Scalable Large Language Model Finetuning. arXiv:2402.07818.

参考文献 61-80

[61] Yihua Zhang et al. (2024). Revisiting Zeroth-Order Optimization for Memory-Efficient LLM Fine-Tuning: A Benchmark. arXiv:2402.11592.

[62] Yong Liu et al. (2024). Sparse MeZO: Less Parameters for Better Performance in Zeroth-Order LLM Fine-Tuning. arXiv:2402.15751.

[63] Robert Tjarko Lange et al. (2024). Large Language Models As Evolution Strategies. arXiv:2402.18381.

[64] Wenyang Hu et al. (2024). Localized Zeroth-Order Prompt Optimization. arXiv:2403.02993.

[65] Tanmay Gautam et al. (2024). Variance-reduced Zeroth-Order Methods for Fine-Tuning Language Models. arXiv:2404.08080.

[66] Yifan Yang et al. (2024). AdaZeta: Adaptive Zeroth-Order Tensor-Train Adaption for Memory-Efficient Large Language Models Fine-Tuning. arXiv:2406.18060.

[67] Zeman Li et al. (2024). Addax: Utilizing Zeroth-Order Gradients to Improve Memory Efficiency and Performance of SGD for Fine-Tuning Language Models. arXiv:2410.06441.

[68] Yiming Chen et al. (2024). Enhancing Zeroth-order Fine-tuning for Language Models with Low-rank Structures. arXiv:2410.07698.

[69] Ziming Yu et al. (2024). Zeroth-Order Fine-Tuning of LLMs in Random Subspaces. arXiv:2410.08989.

[70] Fei Wang et al. (2024). Simultaneous Computation and Memory Efficient Zeroth-Order Optimizer for Fine-Tuning Large Language Models. arXiv:2410.09823.

[71] Qitao Tan et al. (2025). Harmony in Divergence: Towards Fast, Accurate, and Memory-efficient Zeroth-order LLM Fine-tuning. arXiv:2502.03304.

[72] Mohamed Aboelenien Ahmed et al. (2025). Efficient Zeroth-Order Federated Finetuning of Language Models on Resource-Constrained Devices. arXiv:2502.10239.

[73] Zhen Zhang et al. (2025). MaZO: Masked Zeroth-Order Optimization for Multi-Task Fine-Tuning of Large Language Models. arXiv:2502.11513.

[74] Jiajun Zhou et al. (2025). QuZO: Quantized Zeroth-Order Fine-Tuning for Large Language Models. arXiv:2502.12346.

[75] Lunchen Xie et al. (2025). ZO-DARTS++: An Efficient and Size-Variable Zeroth-Order Neural Architecture Search Algorithm. arXiv:2503.06092.

[76] Seonghwan Park et al. (2025). ZIP: An Efficient Zeroth-order Prompt Tuning for Black-box Vision-Language Models. arXiv:2504.06838.

[77] Liang Zhang et al. (2025). Zeroth-Order Optimization Finds Flat Minima. arXiv:2506.05454.

[78] Sizhe Dang et al. (2025). FZOO: Fast Zeroth-Order Optimizer for Fine-Tuning Large Language Models towards Adam-Scale Speed. arXiv:2506.09034.

[79] Haishan Ye et al. (2026). Why Does Adaptive Zeroth-Order Optimization Work?. arXiv:2602.01627.

[80] Mert Cemri et al. (2026). AdaEvolve: Adaptive LLM Driven Zeroth-Order Optimization. arXiv:2602.20133.

参考文献 81-100

[81] Zelin Li et al. (2026). Inference-Native Zeroth-Order Optimization for LLMs. arXiv:2605.28760.

[82] Liyan Tan et al. (2026). GRZO: Group-Relative Zeroth-Order Optimization for Large Language Model Fine-Tuning. arXiv:2606.02857.

[83] Jaemin Kim et al. (2026). VLA-ZO: Fast Zeroth-Order Adaptation for Vision-Language-Action Models. arXiv:2610.06271.

[84] Chuang Tong et al. (2026). Zeroth-Order Fine-Tuning of LLMs in Learnable Low-Rank Subspaces. arXiv:2610.15225.

[85] Xuanlin Yang et al. (2026). StructZO: Structured Zeroth-Order Optimization for Memory-Efficient LLM Fine-Tuning. arXiv:2610.16451.

[86] Lukasz Bartoszcze et al. (2025). Representation Engineering for Large-Language Models: Survey and Research Challenges. arXiv:2502.17601.

[87] Haotian Yang et al. (2026). Does Steering Break Your Model? A Multi-Dimensional Evaluation Suite for LLM Steering Methods. arXiv:2610.07722.

[88] Sumanth Dathathri et al. (2019). Plug and Play Language Models: A Simple Approach to Controlled Text Generation. arXiv:1912.02164.

[89] Nishant Subramani et al. (2022). Extracting Latent Steering Vectors from Pretrained Language Models. arXiv:2205.05124.

[90] Kenneth Li et al. (2023). Inference-Time Intervention: Eliciting Truthful Answers from a Language Model. arXiv:2306.03341.

[91] Alexander Matt Turner et al. (2023). Steering Language Models With Activation Engineering. arXiv:2308.10248.

[92] Hoagy Cunningham et al. (2023). Sparse Autoencoders Find Highly Interpretable Features in Language Models. arXiv:2309.08600.

[93] Andy Zou et al. (2023). Representation Engineering: A Top-Down Approach to AI Transparency. arXiv:2310.01405.

[94] Ole Jorgensen et al. (2023). Improving Activation Steering in Language Models with Mean-Centring. arXiv:2312.03813.

[95] Nina Panickssery et al. (2023). Steering Llama 2 via Contrastive Activation Addition. arXiv:2312.06681.

[96] Yotam Wolf et al. (2024). Tradeoffs Between Alignment and Helpfulness in Language Models with Steering Methods. arXiv:2401.16332.

[97] János Kramár et al. (2024). AtP*: An efficient and scalable method for localizing LLM behaviour to components. arXiv:2403.00745.

[98] Teun van der Weij et al. (2024). Extending Activation Steering to Broad Skills and Multiple Behaviours. arXiv:2403.05767.

[99] Zhengxuan Wu et al. (2024). ReFT: Representation Finetuning for Language Models. arXiv:2404.03592.

[100] Leo Gao et al. (2024). Scaling and evaluating sparse autoencoders. arXiv:2406.04093.

参考文献 101-120

[101] Senthooran Rajamanoharan et al. (2024). Jumping Ahead: Improving Reconstruction Fidelity with JumpReLU Sparse Autoencoders. arXiv:2407.14435.

[102] Bruce W. Lee et al. (2024). Programming Refusal with Conditional Activation Steering. arXiv:2409.05907.

[103] Alessandro Stolfo et al. (2024). Improving Instruction-Following in Language Models through Activation Steering. arXiv:2410.12877.

[104] Sheng Liu et al. (2024). Reducing Hallucinations in Vision-Language Models via Latent Space Steering. arXiv:2410.15778.

[105] Bart Bussmann et al. (2024). BatchTopK Sparse Autoencoders. arXiv:2412.06410.

[106] Gonçalo Paulo et al. (2025). Transcoders Beat Sparse Autoencoders for Interpretability. arXiv:2501.18823.

[107] Jan Wehner et al. (2025). Taxonomy, Opportunities, and Challenges of Representation Engineering for Large Language Models. arXiv:2502.19649.

[108] Zirui He et al. (2025). SAE-SSV: Supervised Steering in Sparse Representation Spaces for Reliable Control of Language Models. arXiv:2505.16188.

[109] Zhengxuan Wu et al. (2025). Improved Representation Steering for Language Models. arXiv:2505.20809.

[110] Jiuding Sun et al. (2025). HyperSteer: Activation Steering at Scale with Hypernetworks. arXiv:2506.03292.

[111] Anton Korznikov et al. (2025). The Rogue Scalpel: Activation Steering Compromises LLM Safety. arXiv:2509.22067.

[112] Claire Tian et al. (2025). Measuring Sparse Autoencoder Feature Sensitivity. arXiv:2509.23717.

[113] Dung V. Nguyen et al. (2025). Activation Steering with a Feedback Controller. arXiv:2510.04309.

[114] Alex Ferrando et al. (2025). Dynamically Scaled Activation Steering. arXiv:2512.03661.

[115] Gracjan Góral et al. (2025). Depth-Wise Activation Steering for Honest Language Models. arXiv:2512.07667.

[116] Hengyuan Zhang et al. (2026). Locate, Steer, and Improve: A Practical Survey of Actionable Mechanistic Interpretability in Large Language Models. arXiv:2601.14004.

[117] Rhitabrat Pokharel et al. (2026). Cross-Lingual Activation Steering for Multilingual Language Models. arXiv:2601.16390.

[118] Zijian Feng et al. (2026). Fine-Grained Activation Steering: Steering Less, Achieving More. arXiv:2602.04428.

[119] Alex McKenzie et al. (2026). Endogenous Resistance to Activation Steering in Language Models. arXiv:2602.06941.

[120] Chung-En Sun et al. (2026). Steer2Edit: From Activation Steering to Component-Level Editing. arXiv:2602.09870.

参考文献 121-140

[121] Aruna Sankaranarayanan et al. (2026). Activation Steering via Generative Causal Mediation. arXiv:2602.16080.

[122] Brandon Hsu et al. (2026). Contextual Linear Activation Steering of Language Models. arXiv:2604.24693.

[123] Tam Nguyen et al. (2026). Minimizing Collateral Damage in Activation Steering. arXiv:2605.01167.

[124] David Chanin (2026). Are Sparse Autoencoder Benchmarks Reliable?. arXiv:2605.18229.

[125] Walter Nelson et al. (2026). Toward Identifiable Sparse Autoencoders. arXiv:2605.31245.

[126] Chong Yong Ong et al. (2026). Forecasting Side Effects of Activation Steering. arXiv:2608.11227.

[127] Zhangdie Yuan et al. (2026). Evaluating the Semantic Specificity of Representation Steering in Language Models. arXiv:2608.29431.

[128] Laziz U. Abdullaev et al. (2026). Kernelized Activation Steering. arXiv:2610.01062.

[129] Seyedarmin Azizi et al. (2026). The Sharp Directions Are Against You: Curvature Analysis of Activation Steering. arXiv:2610.16084.

[130] Nishkal Hundia et al. (2026). Sweep Less, Steer More: Using Activation Steering Geometry for Reduced Hyperparameter Search. arXiv:2611.01496.

[131] Thomas Elsken et al. (2019). Neural Architecture Search: A Survey. *JMLR* 20(55):1-21.

[132] Barret Zoph et al. (2016). Neural Architecture Search with Reinforcement Learning. arXiv:1611.01578.

[133] Chenxi Liu et al. (2017). Progressive Neural Architecture Search. arXiv:1712.00559.

[134] Esteban Real et al. (2018). Regularized Evolution for Image Classifier Architecture Search. arXiv:1802.01548.

[135] Hieu Pham et al. (2018). Efficient Neural Architecture Search via Parameter Sharing. arXiv:1802.03268.

[136] Hanxiao Liu et al. (2018). DARTS: Differentiable Architecture Search. arXiv:1806.09055.

[137] Chris Zhang et al. (2018). Graph HyperNetworks for Neural Architecture Search. arXiv:1810.05749.

[138] Han Cai et al. (2018). ProxylessNAS: Direct Neural Architecture Search on Target Task and Hardware. arXiv:1812.00332.

[139] Sirui Xie et al. (2018). SNAS: Stochastic Neural Architecture Search. arXiv:1812.09926.

[140] Liam Li et al. (2019). Random Search and Reproducibility for Neural Architecture Search. arXiv:1902.07638.

参考文献 141-160

[141] Chris Ying et al. (2019). NAS-Bench-101: Towards Reproducible Neural Architecture Search. arXiv:1902.09635.

[142] Xin Chen et al. (2019). Progressive Differentiable Architecture Search: Bridging the Depth Gap between Search and Evaluation. arXiv:1904.12760.

[143] Martin Wistuba et al. (2019). A Survey on Neural Architecture Search. arXiv:1905.01392.

[144] Yuhui Xu et al. (2019). PC-DARTS: Partial Channel Connections for Memory-Efficient Architecture Search. arXiv:1907.05737.

[145] Han Cai et al. (2019). Once-for-All: Train One Network and Specialize it for Efficient Deployment. arXiv:1908.09791.

[146] Marius Lindauer et al. (2019). Best Practices for Scientific Research on Neural Architecture Search. arXiv:1909.02453.

[147] Arber Zela et al. (2019). Understanding and Robustifying Differentiable Architecture Search. arXiv:1909.09656.

[148] Xuanyi Dong et al. (2020). NAS-Bench-201: Extending the Scope of Reproducible Neural Architecture Search. arXiv:2001.00326.

[149] Joseph Mellor et al. (2020). Neural Architecture Search without Training. arXiv:2006.04647.

[150] Nikita Klyuchnikov et al. (2020). NAS-Bench-NLP: Neural Architecture Search Benchmark for Natural Language Processing. arXiv:2006.07116.

[151] Xiangning Chen et al. (2020). DrNAS: Dirichlet Neural Architecture Search. arXiv:2006.10355.

[152] Arber Zela et al. (2020). Surrogate NAS Benchmarks: Going Beyond the Limited Search Spaces of Tabular NAS Benchmarks. arXiv:2008.09777.

[153] Xuanyi Dong et al. (2020). NATS-Bench: Benchmarking NAS Algorithms for Architecture Topology and Size. arXiv:2009.00437.

[154] Mohamed S. Abdelfattah et al. (2021). Zero-Cost Proxies for Lightweight NAS. arXiv:2101.08134.

[155] Xiaofang Wang et al. (2021). Neighborhood-Aware Neural Architecture Search. arXiv:2105.06369.

[156] Ruochen Wang et al. (2021). Rethinking Architecture Selection in Differentiable NAS. arXiv:2108.04392.

[157] Kaicheng Yu et al. (2021). An Analysis of Super-Net Heuristics in Weight-Sharing NAS. arXiv:2110.01154.

[158] Jingjing Xu et al. (2021). KNAS: Green Neural Architecture Search. arXiv:2111.13293.

[159] Mojan Javaheripi et al. (2022). LiteTransformerSearch: Training-free Neural Architecture Search for Efficient Language Models. arXiv:2203.02094.

[160] Angelica Chen et al. (2023). EvoPrompting: Language Models for Code-Level Neural Architecture Search. arXiv:2302.14838.

参考文献 161-180

[161] Neal Lawton et al. (2023). Neural Architecture Search for Parameter-Efficient Fine-tuning of Large Pre-trained Language Models. arXiv:2305.16597.

[162] Muhammad U. Nasir et al. (2023). LLMatic: Neural Architecture Search via Large Language Models and Quality Diversity Optimization. arXiv:2306.01102.

[163] Yiyang Zhao et al. (2023). Carbon-Efficient Neural Architecture Search. arXiv:2307.04131.

[164] Rhea Sanjay Sukthanker et al. (2024). Multi-objective Differentiable Neural Architecture Search. arXiv:2402.18213.

[165] Aaron Klein et al. (2024). Structural Pruning of Pre-trained Language Models via Neural Architecture Search. arXiv:2405.02267.

[166] Anthony Sarah et al. (2024). LLaMA-NAS: Efficient Neural Architecture Search for Large Language Models. arXiv:2405.18377.

[167] Xun Zhou et al. (2024). Design Principle Transfer in Neural Architecture Search via Large Language Models. arXiv:2408.11330.

[168] J. Pablo Muñoz et al. (2025). Low-Rank Adapters Meet Neural Architecture Search for LLM Compression. arXiv:2501.16372.

[169] Zicheng Cai et al. (2025). SEKI: Self-Evolution and Knowledge Inspiration based Neural Architecture Search via Large Language Models. arXiv:2502.20422.

[170] Zhen-Song Chen et al. (2025). ZeroLM: Data-Free Transformer Architecture Search for Language Models. arXiv:2503.18646.

[171] Shang Wang (2025). Elastic Architecture Search for Efficient Language Models. arXiv:2510.27037.

[172] Seokhoon Jeong et al. (2026). Agentic Neural Architecture Search. arXiv:2607.07984.

[173] Brama Seta Putra Satria et al. (2026). QuaNAS: Uncertainty-Aware Reuse of Earlier Searches for Query-Efficient Neural Architecture Search. arXiv:2610.31251.

[174] Yuefeng Xu et al. (2026). Searching with Sparse Histories: Cross-Task Architecture Search from Incomplete Search Logs and Zero-Cost Proxies. arXiv:2610.32179.

[175] Yize Sun et al. (2026). DIFFNAS: Predictor-Free Neural Architecture Search via Diffusion Model and Zero-Cost Evaluation. arXiv:2610.41818.

[176] Kaichen Zhou et al. (2026). SteerAct: Unlocking Action Models through Few-Shot Temporal Steering. arXiv:2610.25632.

[177] Stephen James et al. (2019). RLBench: The Robot Learning Benchmark & Learning Environment. arXiv:1909.12271.

[178] Anthony Brohan et al. (2022). RT-1: Robotics Transformer for Real-World Control at Scale. arXiv:2212.06817.

[179] Cheng Chi et al. (2023). Diffusion Policy: Visuomotor Policy Learning via Action Diffusion. arXiv:2303.04137.

[180] Bo Liu et al. (2023). LIBERO: Benchmarking Knowledge Transfer for Lifelong Robot Learning. arXiv:2306.03310.

参考文献 181-200

[181] Anthony Brohan et al. (2023). RT-2: Vision-Language-Action Models Transfer Web Knowledge to Robotic Control. arXiv:2307.15818.

[182] Open X-Embodiment Collaboration et al. (2023). Open X-Embodiment: Robotic Learning Datasets and RT-X Models. arXiv:2310.08864.

[183] Octo Model Team et al. (2024). Octo: An Open-Source Generalist Robot Policy. arXiv:2405.12213.

[184] Moo Jin Kim et al. (2024). OpenVLA: An Open-Source Vision-Language-Action Model. arXiv:2406.09246.

[185] Kevin Black et al. (2024). pi_0: A Vision-Language-Action Flow Model for General Robot Control. arXiv:2410.24164.

[186] Maximilian Tölle et al. (2025). Towards Safe Robot Foundation Models. arXiv:2503.07404.

[187] Shuhan Tan et al. (2025). Interactive Post-Training for Vision-Language-Action Models. arXiv:2505.17016.

[188] Thomas P Zollo et al. (2025). Confidence Calibration in Vision-Language-Action Models. arXiv:2507.17383.

[189] Bear Häon et al. (2025). Mechanistic interpretability for steering vision-language-action models. arXiv:2509.00328.

[190] Zhaoshu Yu et al. (2025). A Survey on Efficient Vision-Language-Action Models. arXiv:2510.24795.

[191] Pranav Guruprasad et al. (2025). Benchmarking the Generality of Vision-Language-Action Models. arXiv:2512.11315.

[192] Shuo Liu et al. (2026). VLS: Steering Pretrained Robot Policies via Vision-Language Models. arXiv:2602.03973.

[193] Harold Soh et al. (2026). Action Hallucination in Generative Vision-Language-Action Models. arXiv:2602.06339.

[194] Pablo Valle et al. (2026). Metamorphic Testing of Vision-Language Action-Enabled Robots. arXiv:2602.22579.

[195] Jonathan C. Kao et al. (2026). Flow Control: Steering Vision-Language-Action Models with Simple Real-Time Inputs. arXiv:2606.10180.

[196] Jason Chan et al. (2026). Steering Autoregressive Vision-Language-Action Policies via Action Token Intervention. arXiv:2606.15021.

[197] Yukiyasu Domae et al. (2026). The Embodiment Gap in Robot Foundation Models. arXiv:2608.18433.

[198] Lin Liu et al. (2026). LIBERO-RECOVER: Beyond Task Success Towards Failure Recovery in Robotic Manipulation Models. arXiv:2609.05178.

[199] Huiqiong Li et al. (2026). LIBERO-VPro: Benchmarking Closed-Loop Visual Robustness of Robotic Foundation Models. arXiv:2609.24350.

[200] Shaojie Ren et al. (2026). LIBERO-Dynamic: Benchmarking Robotic Manipulation in Dynamic Environments. arXiv:2610.24579.